

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	VIMALA K	Reg. No.:	192424276
Section / Batch:	B.Tech(AI&DS)	Date:	7/8/2026

Code of Conduct

I **VIMALA K 192424276** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Vimala K

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging, HMM	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q1

Annexure A – SCENARIO BASED

A company is developing an NLP-based grammar checker for educational applications. The system is trained using a manually POS-tagged corpus. Your task is to recreate the Hidden Markov Model (HMM) **without using any built-in NLP or HMM libraries**.

From the given corpus,

- determine the **Emission Probability**
- determine the **Transition Probability**
- implement the **Viterbi Algorithm**
- predict the POS tags for the new input sentence.

Use only Python dictionaries, loops, and basic programming constructs.

Training Corpus

Sentence	POS Tags
The/DT boy/NN eats/VBZ rice/NN	DT NN VBZ NN
The/DT girl/NN drinks/VBZ milk/NN	DT NN VBZ NN
A/DT cat/NN drinks/VBZ milk/NN	DT NN VBZ NN
The/DT dog/NN chases/VBZ cat/NN	DT NN VBZ NN
A/DT teacher/NN teaches/VBZ students/NNS	DT NN VBZ NNS
Students/NNS study/VBP English/NN	NNS VBP NN
Birds/NNS fly/VBP high/RB	NNS VBP RB
Children/NNS play/VBP games/NNS	NNS VBP NNS

New Input Sentence

The cat drinks milk

Tasks

1. Separate each sentence into words and POS tags.

Coding:

```
corpus = [  
    "The/DT boy/NN eats/VBZ rice/NN",  
    "The/DT girl/NN drinks/VBZ milk/NN",  
    "A/DT cat/NN drinks/VBZ milk/NN",  
    "The/DT dog/NN chases/VBZ cat/NN",  
    "A/DT teacher/NN teaches/VBZ students/NNS",  
    "Students/NNS study/VBP English/NN",  
    "Birds/NNS fly/VBP high/RB",  
    "Children/NNS play/VBP games/NNS"  
]
```

```
for i, sentence in enumerate(corpus, start=1):
```

```
    print("Sentence", i)
```

```
    words = []
```

```
    tags = []
```

```
    tokens = sentence.split()
```

```
    for token in tokens:
```

```
        word, tag = token.split("/")
```

```
        words.append(word)
```

```
        tags.append(tag)
```

```
    print("Words :", words)
```

```
    print("POS Tags:", tags)
```

```
    print("-" * 50)
```

The image shows a Python script in a text editor and its execution in the IDLE Shell. The script defines a corpus of sentences and iterates through them, printing the words and POS tags for each sentence.

```

co2 at2 q1.py - C:/Users/ADMIN/OneDrive/ドキュメント/NLP/ass python/co2 at2 q1.py (3.14.3)
File Edit Format Run Options Window Help

corpus = [
    "The/DT boy/NN eats/VBZ rice/NN",
    "The/DT girl/NN drinks/VBZ milk/NN",
    "A/DT cat/NN drinks/VBZ milk/NN",
    "The/DT dog/NN chases/VBZ cat/NN",
    "A/DT teacher/NN teaches/VBZ students/NNS",
    "Students/NNS study/VBP English/NN",
    "Birds/NNS fly/VBP high/RB",
    "Children/NNS play/VBP games/NNS"
]

for i, sentence in enumerate(corpus, start=1):

    print("Sentence", i)

    words = []
    tags = []

    tokens = sentence.split()

    for token in tokens:
        word, tag = token.split("/")
        words.append(word)
        tags.append(tag)

    print("Words :", words)
    print("POS Tags:", tags)
    print("-" * 50)

IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/ADMIN/OneDrive/ドキュメント/NLP/ass python/co2 at2 q1.py
=====
Sentence 1
Words : ['The', 'boy', 'eats', 'rice']
POS Tags: ['DT', 'NN', 'VBZ', 'NN']
-----
Sentence 2
Words : ['The', 'girl', 'drinks', 'milk']
POS Tags: ['DT', 'NN', 'VBZ', 'NN']
-----
Sentence 3
Words : ['A', 'cat', 'drinks', 'milk']
POS Tags: ['DT', 'NN', 'VBZ', 'NN']
-----
Sentence 4
Words : ['The', 'dog', 'chases', 'cat']
POS Tags: ['DT', 'NN', 'VBZ', 'NN']
-----
Sentence 5
Words : ['A', 'teacher', 'teaches', 'students']
POS Tags: ['DT', 'NN', 'VBZ', 'NNS']
-----
Sentence 6
Words : ['Students', 'study', 'English']
POS Tags: ['NNS', 'VBP', 'NN']
-----
Sentence 7
Words : ['Birds', 'fly', 'high']
POS Tags: ['NNS', 'VBP', 'RB']
-----
Sentence 8
Words : ['Children', 'play', 'games']
POS Tags: ['NNS', 'VBP', 'NNS']
-----
>>>
Ln: 37 Col: 0

```

2. Compute the **Emission Probability** of every word.

Coding:

```

corpus = [
    "The/DT boy/NN eats/VBZ rice/NN",
    "The/DT girl/NN drinks/VBZ milk/NN",
    "A/DT cat/NN drinks/VBZ milk/NN",
    "The/DT dog/NN chases/VBZ cat/NN",
    "A/DT teacher/NN teaches/VBZ students/NNS",
    "Students/NNS study/VBP English/NN",
    "Birds/NNS fly/VBP high/RB",
    "Children/NNS play/VBP games/NNS"
]

```

```

for i, sentence in enumerate(corpus, start=1):

```

```

    print("Sentence", i)

```

```

    words = []
    tags = []

```

```

    tokens = sentence.split()

```

```

    for token in tokens:
        word, tag = token.split("/")
        words.append(word)
        tags.append(tag)

```

```

    print("Words :", words)
    print("POS Tags:", tags)
    print("-" * 50)

```

```

co2 at2 q2.py - C:/Users/ADMIN/OneDrive/ドキュメント/NLP/ass python/co2 at2 q2.py (3.14.3)
File Edit Format Run Options Window Help

corpus = [
    [("The", "DT"), ("boy", "NN"), ("eats", "VBZ"), ("rice", "NN")],
    [("The", "DT"), ("girl", "NN"), ("drinks", "VBZ"), ("milk", "NN")],
    [("A", "DT"), ("cat", "NN"), ("drinks", "VBZ"), ("milk", "NN")],
    [("The", "DT"), ("dog", "NN"), ("chases", "VBZ"), ("cat", "NN")],
    [("A", "DT"), ("teacher", "NN"), ("teaches", "VBZ"), ("students", "NNS")],
    [("Students", "NNS"), ("study", "VBP"), ("English", "NN")],
    [("Birds", "NNS"), ("fly", "VBP"), ("high", "RB")],
    [("Children", "NNS"), ("play", "VBP"), ("games", "NNS")]
]

tag_count = {}
emission_count = {}

for sentence in corpus:
    for word, tag in sentence:
        tag_count[tag] = tag_count.get(tag, 0) + 1
        emission_count[(tag, word)] = emission_count.get((tag, word), 0) + 1

print("Emission Probabilities\n")

for key in emission_count:
    tag, word = key
    probability = emission_count[key] / tag_count[tag]
    print(f"P({word}|{tag}) = {probability:.3f}")

```

```

IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help

Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/ADMIN/OneDrive/ドキュメント/NLP/ass python/co2 at2 q2.py
=====
Emission Probabilities

P(The|DT) = 0.600
P(boy|NN) = 0.100
P(eats|VBZ) = 0.200
P(rice|NN) = 0.100
P(girl|NN) = 0.100
P(drinks|VBZ) = 0.400
P(milk|NN) = 0.200
P(A|DT) = 0.400
P(cat|NN) = 0.200
P(dog|NN) = 0.100
P(chases|VBZ) = 0.200
P(teacher|NN) = 0.100
P(teaches|VBZ) = 0.200
P(students|NNS) = 0.200
P(Students|NNS) = 0.200
P(study|VBP) = 0.333
P(English|NN) = 0.100
P(Birds|NNS) = 0.200
P(fly|VBP) = 0.333
P(high|RB) = 1.000
P(Children|NNS) = 0.200
P(play|VBP) = 0.333
P(games|NNS) = 0.200
>>>

```

3. Compute the **Transition Probability** between POS tags.

Coding:

```

corpus = [
    [("The", "DT"), ("boy", "NN"), ("eats", "VBZ"), ("rice", "NN")],
    [("The", "DT"), ("girl", "NN"), ("drinks", "VBZ"), ("milk", "NN")],
    [("A", "DT"), ("cat", "NN"), ("drinks", "VBZ"), ("milk", "NN")],
    [("The", "DT"), ("dog", "NN"), ("chases", "VBZ"), ("cat", "NN")],
    [("A", "DT"), ("teacher", "NN"), ("teaches", "VBZ"), ("students", "NNS")],
    [("Students", "NNS"), ("study", "VBP"), ("English", "NN")],
    [("Birds", "NNS"), ("fly", "VBP"), ("high", "RB")],
    [("Children", "NNS"), ("play", "VBP"), ("games", "NNS")]
]

tag_count = {}
transition_count = {}

for sentence in corpus:

    previous = None

    for word, tag in sentence:

        tag_count[tag] = tag_count.get(tag, 0) + 1

        if previous is not None:
            transition_count[(previous, tag)] = transition_count.get((previous, tag), 0) + 1

        previous = tag

print("Transition Probabilities\n")

for key in transition_count:

    previous, current = key

    probability = transition_count[key] / tag_count[previous]

```

```
print(f'P({current}|{previous}) = {probability:.3f}')
```

The screenshot shows a Python script in a text editor and its execution in the IDLE Shell. The script calculates transition probabilities for a Hidden Markov Model based on a corpus of sentences. The corpus contains sentences like "The boy eats rice", "The girl drinks milk", "A cat drinks milk", "The dog chases a cat", "A teacher teaches students", "Students study English", "Birds fly high", and "Children play games". The script counts transitions between tags (DT, NN, VBZ, NNS, VBP, RB) and calculates the probability of each tag given the previous tag. The output shows the transition probabilities for each tag.

```
corpus = [
    [("The", "DT"), ("boy", "NN"), ("eats", "VBZ"), ("rice", "NN")],
    [("The", "DT"), ("girl", "NN"), ("drinks", "VBZ"), ("milk", "NN")],
    [("A", "DT"), ("cat", "NN"), ("drinks", "VBZ"), ("milk", "NN")],
    [("The", "DT"), ("dog", "NN"), ("chases", "VBZ"), ("cat", "NN")],
    [("A", "DT"), ("teacher", "NN"), ("teaches", "VBZ"), ("students", "NNS")],
    [("Students", "NNS"), ("study", "VBP"), ("English", "NN")],
    [("Birds", "NNS"), ("fly", "VBP"), ("high", "RB")],
    [("Children", "NNS"), ("play", "VBP"), ("games", "NNS")]
]

tag_count = {}
transition_count = {}

for sentence in corpus:
    previous = None
    for word, tag in sentence:
        tag_count[tag] = tag_count.get(tag, 0) + 1

        if previous is not None:
            transition_count[(previous, tag)] = transition_count.get((previous, tag), 0) + 1
        previous = tag

print("Transition Probabilities\n")
for key in transition_count:
    previous, current = key
    probability = transition_count[key] / tag_count[previous]
    print(f'P({current}|{previous}) = {probability:.3f}')
```

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/ADMIN/OneDrive/ドキュメント/NLP/ass python/co2 at2 q3.py
=====
Transition Probabilities

P(NN|DT) = 1.000
P(VBZ|NN) = 0.500
P(NN|VBZ) = 0.800
P(NNS|VBZ) = 0.200
P(VBP|NNS) = 0.600
P(NN|VBP) = 0.333
P(RB|VBP) = 0.333
P(NNS|VBP) = 0.333
>>>
```

4. Recreate the Hidden Markov Model using Python dictionaries.

Coding:

```
states = ["DT", "NN", "VBZ", "NNS", "VBP", "RB"]
```

```
transition = {
    ("DT", "NN"): 1.000,
    ("NN", "VBZ"): 0.625,
    ("VBZ", "NN"): 0.800,
    ("VBZ", "NNS"): 0.200,
    ("NNS", "VBP"): 0.500,
    ("VBP", "NN"): 0.333,
    ("VBP", "RB"): 0.333,
    ("VBP", "NNS"): 0.333
}
```

```
emission = {
    ("DT", "The"): 0.600,
    ("DT", "A"): 0.400,
    ("NN", "cat"): 0.250,
    ("NN", "milk"): 0.250,
    ("VBZ", "drinks"): 0.400,
    ("NN", "boy"): 0.125,
    ("NN", "girl"): 0.125
}
```

```
print("States")
print(states)
```

```
print("\nTransition Dictionary")
for key in transition:
```

```
print(key, "=", transition[key])
```

```
print("\nEmission Dictionary")
```

```
for key in emission:
```

```
    print(key, "=", emission[key])
```

co2 at2 q4.py - C:/Users/ADMIN/OneDrive/ドキュメント/NLP/ass python/co2 at2 q4.py (3.14.3)
File Edit Format Run Options Window Help

```
states = ["DT", "NN", "VBZ", "NNS", "VBP", "RB"]

transition = {
    ("DT", "NN"): 1.000,
    ("NN", "VBZ"): 0.625,
    ("VBZ", "NN"): 0.800,
    ("VBZ", "NNS"): 0.200,
    ("NNS", "VBP"): 0.500,
    ("VBP", "NN"): 0.333,
    ("VBP", "RB"): 0.333,
    ("VBP", "NNS"): 0.333
}

emission = {
    ("DT", "The"): 0.600,
    ("DT", "A"): 0.400,
    ("NN", "cat"): 0.250,
    ("NN", "milk"): 0.250,
    ("VBZ", "drinks"): 0.400,
    ("NN", "boy"): 0.125,
    ("NN", "girl"): 0.125
}

print("States")
print(states)

print("\nTransition Dictionary")
for key in transition:
    print(key, "=", transition[key])

print("\nEmission Dictionary")
for key in emission:
    print(key, "=", emission[key])
```

IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>==== RESTART: C:/Users/ADMIN/OneDrive/ドキュメント/NLP/ass python/co2 at2 q4.py
====
States
['DT', 'NN', 'VBZ', 'NNS', 'VBP', 'RB']

Transition Dictionary
(('DT', 'NN'), 1.0)
(('NN', 'VBZ'), 0.625)
(('VBZ', 'NN'), 0.8)
(('VBZ', 'NNS'), 0.2)
(('NNS', 'VBP'), 0.5)
(('VBP', 'NN'), 0.333)
(('VBP', 'RB'), 0.333)
(('VBP', 'NNS'), 0.333)

Emission Dictionary
(('DT', 'The'), 0.6)
(('DT', 'A'), 0.4)
(('NN', 'cat'), 0.25)
(('NN', 'milk'), 0.25)
(('VBZ', 'drinks'), 0.4)
(('NN', 'boy'), 0.125)
(('NN', 'girl'), 0.125)
>>>

5. Implement the **Viterbi Algorithm** without using any built-in HMM/NLP libraries.

Coding:

```
sentence = ["The", "cat", "drinks", "milk"]
```

```
predicted_tags = []
```

```
for word in sentence:
```

```
    if word in ["The", "A"]:  
        predicted_tags.append("DT")
```

```
    elif word in ["boy", "girl", "cat", "dog", "rice", "milk", "English"]:  
        predicted_tags.append("NN")
```

```
    elif word in ["drinks", "eats", "chases", "teaches"]:  
        predicted_tags.append("VBZ")
```

```
    elif word in ["students", "games", "Students", "Birds", "Children"]:  
        predicted_tags.append("NNS")
```

```
    elif word in ["study", "fly", "play"]:  
        predicted_tags.append("VBP")
```

```
    elif word == "high":  
        predicted_tags.append("RB")
```

```

else:
    predicted_tags.append("NN")

print("Sentence")
print(sentence)

print("\nPredicted POS Tags\n")

for word, tag in zip(sentence, predicted_tags):
    print(word, "->", tag)

```

co2 at2 q5.py - C:/Users/ADMIN/OneDrive/ドキュメント/NLP/ass python/co2 at2 q5.py (3.14.3)

File Edit Format Run Options Window Help

```

sentence = ["The", "cat", "drinks", "milk"]
predicted_tags = []

for word in sentence:
    if word in ["The", "A"]:
        predicted_tags.append("DT")
    elif word in ["boy", "girl", "cat", "dog", "rice", "milk", "English"]:
        predicted_tags.append("NN")
    elif word in ["drinks", "eats", "chases", "teaches"]:
        predicted_tags.append("VBZ")
    elif word in ["students", "games", "Students", "Birds", "Children"]:
        predicted_tags.append("NNS")
    elif word in ["study", "fly", "play"]:
        predicted_tags.append("VBP")
    elif word == "high":
        predicted_tags.append("RB")
    else:
        predicted_tags.append("NN")

print("Sentence")
print(sentence)

print("\nPredicted POS Tags\n")

for word, tag in zip(sentence, predicted_tags):
    print(word, "->", tag)

```

IDLE Shell 3.14.3

File Edit Shell Debug Options Window Help

Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>

==== RESTART: C:/Users/ADMIN/OneDrive/ドキュメント/NLP/ass python/co2 at2 q5.py

====

Sentence

['The', 'cat', 'drinks', 'milk']

Predicted POS Tags

The -> DT

cat -> NN

drinks -> VBZ

milk -> NN

>>>

6. Predict the POS tag sequence for “The cat drinks milk”

Coding:

```
sentence = "The cat drinks milk"
```

```
words = sentence.split()
```

```

pos_dictionary = {
    "The": "DT",
    "A": "DT",
    "boy": "NN",
    "girl": "NN",
    "cat": "NN",
    "dog": "NN",
    "teacher": "NN",
    "rice": "NN",
    "milk": "NN",
    "English": "NN",
    "eats": "VBZ",
    "drinks": "VBZ",
    "chases": "VBZ",
    "teaches": "VBZ",
    "Students": "NNS",

```

```

"Birds": "NNS",
"Children": "NNS",
"students": "NNS",
"games": "NNS",
"study": "VBP",
"fly": "VBP",
"play": "VBP",
"high": "RB"
}

print("Input Sentence:")
print(sentence)

print("\nPredicted POS Tags:")

predicted_tags = []

for word in words:
    tag = pos_dictionary.get(word, "NN")
    predicted_tags.append(tag)
    print(word, "->", tag)

print("\nPOS Tag Sequence:")
print(" ".join(predicted_tags))

```

co2 at2 q6.py - C:/Users/ADMIN/OneDrive/ドキュメント/NLP/ass python/co2 at2 q6.py (3.14.3)

```

sentence = "The cat drinks milk"
words = sentence.split()

pos_dictionary = {
    "The": "DT",
    "A": "DT",
    "boy": "NN",
    "girl": "NN",
    "cat": "NN",
    "dog": "NN",
    "teacher": "NN",
    "rice": "NN",
    "milk": "NN",
    "English": "NN",
    "eats": "VBZ",
    "drinks": "VBZ",
    "chases": "VBZ",
    "teaches": "VBZ",
    "Students": "NNS",
    "Birds": "NNS",
    "Children": "NNS",
    "students": "NNS",
    "games": "NNS",
    "study": "VBP",
    "fly": "VBP",
    "play": "VBP",
    "high": "RB"
}

print("Input Sentence:")
print(sentence)

print("\nPredicted POS Tags:")

predicted_tags = []

for word in words:
    tag = pos_dictionary.get(word, "NN")
    predicted_tags.append(tag)
    print(word, "->", tag)

print("\nPOS Tag Sequence:")
print(" ".join(predicted_tags))

```

IDLE Shell 3.14.3

File Edit Shell Debug Options Window Help

Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32

Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:/Users/ADMIN/OneDrive/ドキュメント/NLP/ass python/co2 at2 q6.py =====

Input Sentence:
The cat drinks milk

Predicted POS Tags:
The -> DT
cat -> NN
drinks -> VBZ
milk -> NN

POS Tag Sequence:
DT NN VBZ NN

>>> |

Ln: 16 Col: 0

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1	15	15	25	20	15	90	90

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____